

Proyecto 1. Formulación Galerkin: Matrices de elementos:

Sebastian Matamala, Nicolas Jimenez, Diego Rojas, Arturo Reyes

9 de julio de 2018

1 Resolución

Al correr el programa se obtiene el siguiente sistema de ecuaciones lineales

$$\begin{bmatrix} 0.014 & -0.014 & 0.0 & 0.0 \\ -0.014 & 0.028 & -0.014 & 0.0 \\ 0.0 & -0.014 & 0.028 & -0.014 \\ 0.0 & 0.0 & -0.014 & 0.014 \end{bmatrix} * \begin{bmatrix} \Phi_0 \\ \Phi_1 \\ \Phi_2 \\ \Phi_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Figure 1: Matriz de sistema lineal

Y resolviendo el sistema de ecuaciones lineales usando los valores de D, L y frontera por defecto se obtiene:

$$\begin{bmatrix} \Phi_0 = 20 \\ \Phi_1 = 20 \\ \Phi_2 = 20 \\ \Phi_3 = -15 \end{bmatrix}$$

Figure 2: Vector de Solucion

2 Programación

2.1 Descripción General

Este informe describira el programa realizado para solucionar el problema descrito en el enunciado.

El programa consta de varias funciones que en conjunto sirven para solucionar un problema de elementos finitos en una dimension.

2.2 Función Elemento

El siguiente segmento de código corresponde a la función para crear las funciones de cada elemento:

```
def elementos():
    for k in range(largo):
        for i in range(2):
            for j in range(2):
                if(i==j):
                    I[i][j] = D[k]/L[k]
                else:
                    I[i][j] = -(D[k]/L[k])
    M.append(I)
```

Esta función calcula cada ecuación de cada elemento y las inserta en un arreglo.

Las siguientes figuras describen la representación matricial de los elementos de la figura.

$$R^{(e)} = \begin{bmatrix} \Phi_i \\ \Phi_j \end{bmatrix} + \frac{D}{L} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \left\{ \begin{bmatrix} \Phi_i \\ \Phi_j \end{bmatrix} \right\} * \frac{QL}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad (1)$$

Figure 3: Función de elemento

e	i	j	D/L	QL/2
1	1	2	1/65	0
2	2	3	1/1600	0
3	3	4	7/500	0

Table 1: Tabla de valores nodales

$$[k^1] = \begin{bmatrix} \frac{1}{65} & -\frac{1}{65} \\ -\frac{1}{65} & \frac{1}{65} \end{bmatrix}, \quad \{f\} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} \quad (2)$$

$$[k^2] = \begin{bmatrix} \frac{1}{1600} & -\frac{1}{1600} \\ -\frac{1}{1600} & \frac{1}{1600} \end{bmatrix}, \quad \{f\} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} \quad (3)$$

$$[k^3] = \begin{bmatrix} \frac{7}{500} & -\frac{7}{500} \\ -\frac{7}{500} & \frac{7}{500} \end{bmatrix}, \quad \{f\} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} \quad (4)$$

Figure 4: Funciones de forma de cada elemento

2.3 Función Global

La funcion que genera la matriz global es :

```
def armaMatriz(A):
    MatrizComp = [[0.0 for x in xrange(len(A)+1)] for x in xrange(len(A)+1)]
    for i in range(0,len(A)):
        for j in range(0,2):
            for k in range(0,2):
                MatrizComp[i+j][i+k] = MatrizComp[i+j][i+k] + A[i][j][k]
    return MatrizComp
```

Esta funcion realiza la sumatoria de todas las funciones en la matriz global.

2.4 Función Valores Nodales

La siguiente seccion de codigo corresponde a la funcion que calcula los valores nodales con los parametros especificados anteriormente

```
def solucion():
    cp = [x for x in V]
    sol = solve(W[0].subs(V[0],20))
    V[0] = FronteraA
    V[1] = sol[0]+F[1]
    print(V[0])
    print(W)
    for i in range(1,largo-1):
        sol = solve(W[i].subs(cp[i-1],V[i-1]).subs(cp[i],V[i]).subs(cp[i+1],V[i+1]))
        V[i+1]= sol[0]+F[i+2]
    V[largo]=FronteraB
    print(V)
```

Esta funcion toma el arreglo de funciones obtenido anteriormente y calcula la solucion al sistema de ecuaciones.

$$\begin{bmatrix} 0.014 & -0.014 & 0.0 & 0.0 \\ -0.014 & 0.028 & -0.014 & 0.0 \\ 0.0 & -0.014 & 0.028 & -0.014 \\ 0.0 & 0.0 & -0.014 & 0.014 \end{bmatrix} * \begin{bmatrix} \Phi_0 \\ \Phi_1 \\ \Phi_2 \\ \Phi_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Figure 5: Matriz de sistema lineal

Finalmente el resultado final de la funcion es:

$$\begin{bmatrix} \Phi_0 = 20 \\ \Phi_1 = 20 \\ \Phi_2 = 20 \\ \Phi_3 = -15 \end{bmatrix}$$

Figure 6: Vector de Solucion

2.5 Función Conductividad

El resultado de multiplicar esta matriz por el vector de incognitas es la siguiente matriz: Estos resultados se obtienen usando como condiciones iniciales de frontera iguales a 0 y los otros valores por defecto Finalmente, esta matriz se

$$\begin{bmatrix} 0.014 * \Phi_0 - 0.014 * \Phi_1 \\ -0.014 * \Phi_0 + 0.028 * \Phi_1 - 0.014 \Phi_2 \\ -0.014 * \Phi_1 + 0.028 * \Phi_2 - 0.014 * \Phi_3 \\ -0.014 * \Phi_2 + 0.014 * \Phi_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Figure 7: Matriz de solucion de ecuaciones lineales

resuelve utilizando sustitucion progresiva y los siguientes valores por defecto.

$$\Phi_0 = 20 \tag{5}$$

$$\Phi_3 = -15 \tag{6}$$